

YOLOv5 를 통한 축구 경기 객체 탐지 프로젝트 보고서

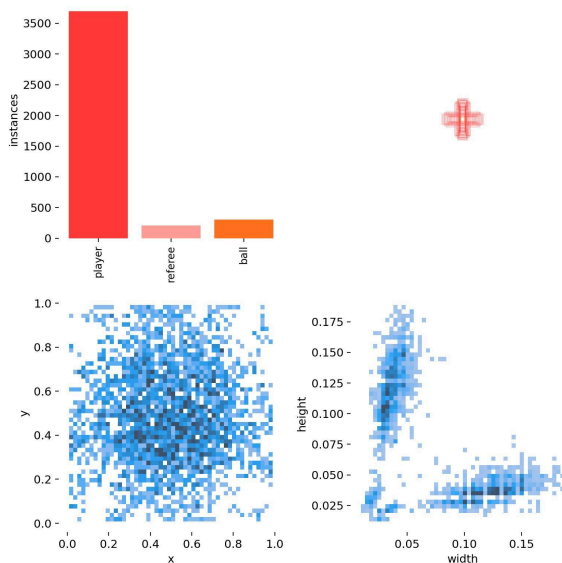
1. 데이터셋 소개

축구 경기 영상의 축구선수와 심판, 공을 식별하기 위해 **futbol players** 라는 데이터셋을 사용했습니다.

- 제작자: Ilyes Talbi (La revue IA)
- 목표: 축구 선수(**player**), 심판(**referee**), 그리고 공(**ball**)을 식별하는 것.
- 클래스 구성:
 - **Class 0**: player
 - **Class 1**: referee
 - **Class 2**: ball
- 특징: Roboflow 플랫폼을 통해 제공되며, 원본 이미지 외에도 다양한 데이터 증강(Augmentation) 및 전처리(Preprocessing)가 적용된 여러 버전이 존재합니다.

아래는 실제 훈련에 사용된 데이터셋의 분포를 시각화한 자료입니다.

labels.jpg 차트를 통해 몇 가지 알 수 있습니다.



- 클래스 불균형 (**Class Imbalance**): **player** 클래스의 인스턴스(객체 수)가 3,500개가 넘는 반면, **referee**와 **ball**은 각각 500개 미만으로 데이터가 매우 불균형합니다. 이는 모델이 **player** 클래스는 잘 학습하지만, 다른 두 클래스는 학습하기 어려울 수 있음을 시사합니다.
- 객체 분포: **x, y** 좌표 히스토그램을 보면 객체들이 이미지 전체에 비교적 고르게 분포되어 있습니다.

2. 모델 및 하이퍼파라미터 설정 ⚙️

이번 실습에서는 YOLOv5 모델 중 **YOLOv5m**을 기반으로 미세조정(Fine-tuning)을 진행했습니다.

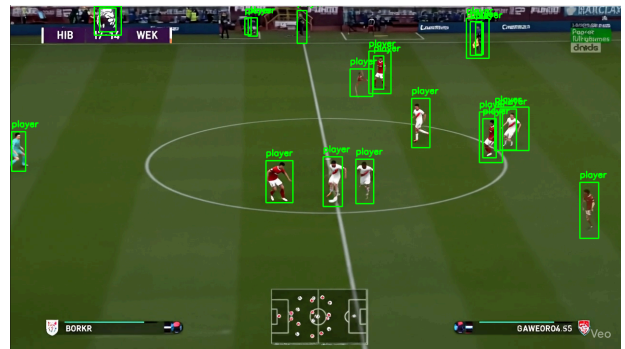
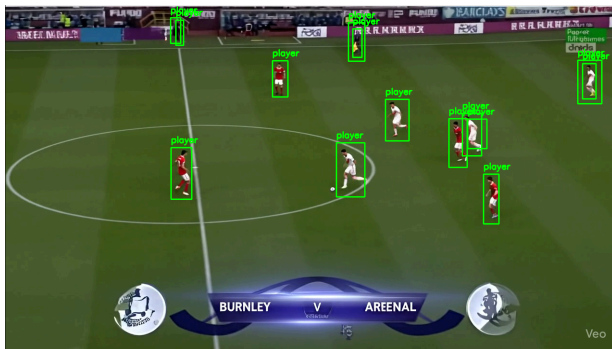
주요 하이퍼파라미터는 다음과 같이 설정되었습니다.

- 기반 모델 (Weights): **yolo5m.pt** (사전 훈련된 COCO 데이터셋 가중치 사용)
- 이미지 크기 (Image Size): **640x640**
- 배치 크기 (Batch Size): **32**
- 에포크 (Epochs): **7**

3. 훈련 결과 분석 📈

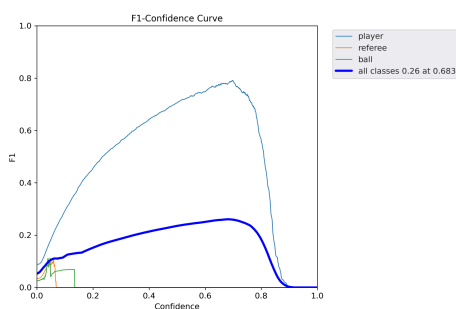
총 7번의 에포크(Epoch) 동안 훈련을 진행했으며, 최종 검증(Validation) 결과는 다음과 같습니다.

- 전체 성능 (all classes):
 - **mAP@0.5: 0.319**
 - **mAP@0.5:0.95: 0.144**
- 클래스별 성능:
 - **player**: 정밀도(P) **0.702**, 재현율(R) **0.877**, mAP@0.5 **0.837**로 상대적으로 우수한 성능을 보였습니다.
 - **referee & ball**: 정밀도(P)는 1이지만 **재현율(R)이 0**으로



나타났습니다. 이는 모델이 심판과 공을 단 한 번도 탐지해내지 못했음을 의미합니다.

결과 분석



- **F1-Confidence Curve**: 위 그래프는 분석 결과를 명확히 보여줍니다. **player** 클래스의 F1 점수는 특정 신뢰도(Confidence) 구간에서 0.8에 가까운 높은 수치를 기록했지만, **referee**와 **ball** 클래스는 F1 점수가 거의 0에 머물러 있습니다.

4. 추론(Inference) 결과 예시

훈련된 모델(best.pt)을 사용하여 실제 축구 영상에 적용한 결과는 다음과 같습니다.

- 성공적인 점: 훈련 결과에서 예측했 듯, 모델은 경기장 내의 **player**들을 매우 안정적으로 탐지해내고 있습니다.
- 한계점: 역시 훈련 결과와 마찬가지로, 영상에 심판이나 공이 명확하게 보임에도 불구하고 전혀 탐지하지 못하는 모습을 확인할 수 있습니다.

5. 결론 및 향후 개선 방안

결론: 이번 YOLOv5 미세조정 실습을 통해 **player** 클래스에 대해서는 준수한 성능의 객체 탐지 모델을 만들었지만, **referee**와 **ball** 클래스는 데이터 불균형 문제로 인해 전혀 탐지하지 못하는 한계를 보였습니다.

향후 개선 방안:

1. 데이터 보강: 가장 시급한 문제로, **referee**와 **ball** 클래스의 데이터를 대량으로 추가 수집하거나, 기존 데이터를 복사하고 변형하는 증강(Augmentation) 기법을 적극적으로 활용해야 합니다.
2. 클래스 가중치 조절: 훈련 시 소수 클래스의 오류에 더 큰 패널티를 부여하는 방식으로 클래스 가중치(Class Weights)를 조정하여 모델이 소수 클래스에 더 집중하도록 유도할 수 있습니다.
3. 추가 훈련: 7 에포크는 모델이 충분히 학습하기에 짧은 시간일 수 있습니다. 더 많은 에포크를 진행하며 성능 변화를 관찰할 필요가 있습니다.
4. 하이퍼파라미터 튜닝: 학습률(Learning Rate), 옵티마이저(Optimizer) 등 다른 하이퍼파라미터를 조정하며 최적의 조합을 찾는 과정이 필요합니다.

```
Object 7: player at [1024, 181, 1048, 262]
Object 8: player at [740, 137, 777, 228]
Object 9: player at [730, 341, 786, 461]
Object 10: player at [897, 56, 920, 127]
Object 11: player at [261, 229, 234, 286]
Object 12: player at [105, 212, 251, 299]
Object 13: player at [1008, 168, 1082, 278]
Object 14: player at [1257, 110, 1280, 165]
Object 15: player at [727, 351, 759, 448]
Object 0: player at [1076, 362, 1132, 466]
Object 1: player at [750, 144, 780, 224]
Object 2: player at [583, 328, 653, 454]
Object 3: player at [1251, 104, 1278, 177]
Object 4: player at [806, 165, 830, 243]
Object 5: player at [1819, 383, 1849, 382]
Object 6: player at [991, 224, 1042, 319]
Object 7: player at [985, 0, 1087, 52]
Object 8: player at [722, 244, 780, 454]
Object 9: player at [898, 56, 920, 127]
Object 10: player at [261, 229, 234, 286]
Object 11: player at [727, 145, 779, 228]
Object 12: player at [1009, 166, 1059, 276]
Object 13: player at [105, 212, 252, 298]
Object 14: player at [584, 343, 638, 458]
Object 15: player at [971, 2, 1015, 50]
Object 0: player at [1072, 364, 1131, 467]
Object 1: player at [751, 144, 772, 225]
Object 2: player at [589, 331, 646, 448]
Object 3: player at [1013, 181, 1044, 261]
Object 4: player at [766, 346, 817, 448]
Object 5: player at [920, 223, 1043, 320]
Object 6: player at [805, 165, 840, 243]
Object 7: player at [1252, 105, 1279, 177]
Object 8: player at [986, 0, 1087, 51]
Object 9: player at [897, 57, 919, 126]
Object 10: player at [737, 149, 784, 225]
Object 11: player at [260, 219, 235, 287]
Object 12: player at [741, 347, 785, 445]
Object 13: player at [1260, 107, 1272, 192]
Object 14: player at [971, 1, 1016, 57]
Object 0: player at [1072, 359, 1129, 467]
Object 1: player at [751, 146, 775, 225]
Object 2: player at [589, 333, 630, 446]
Object 3: player at [995, 231, 1026, 316]
Object 4: player at [714, 347, 810, 448]
Object 5: player at [804, 166, 840, 243]
Object 6: player at [1809, 383, 1842, 383]
Object 7: player at [986, 0, 1088, 51]
Object 8: player at [1252, 104, 1279, 177]
Object 9: player at [897, 57, 920, 126]
Object 10: player at [261, 218, 242, 288]
Object 11: player at [740, 137, 787, 232]
Object 12: player at [742, 351, 787, 444]
Object 13: player at [1262, 105, 1271, 195]
```

```
python train.py --img 640 --batch 16 --epochs 100 --data ./data.yaml --weights yolov5m.pt --device cuda:0 --name train_yolov5m_xpus --save_dir ./runs/train_yolov5m_xpus
```

Model summary: 212 layers, 20861916 parameters, 0 gradients, 47.9 GFLOPs

Class	Images	Instances	P	R	mAP50	mAP50-95
all	33	407	0.903	0.292	0.319	0.144
player	33	359	0.702	0.877	0.837	0.388
referee	33	20	1	0	0.0595	0.0319
ball	33	25	1	0	0.0596	0.0110

```
7 epochs completed in 0.638 hours.
Optimizer stripped from yolov5/runs/train_yolov5m_xpus/weights/last.pt, 84.0MB
Optimizer stripped from yolov5/runs/train_yolov5m_xpus/weights/best.pt, 84.0MB
Validating yolov5/runs/train_yolov5m_xpus/weights/best.pt...
Fusing layers...
Results saved to yolov5/runs/train_yolov5m_xpus
(.venv) xpu@intel:~/Downloads/qwer/bran/yolo/yolo_prac/r020ra-yolov5-finetune-juhuk-1225/cards
```